

CS409m: Introduction to Cryptography

Lecture 07 — Computational Secrecy from PRGs, and How to Actually Build a PRG

Compiled study notes, expanded from the handwritten proof notes
and the lecture slides (Instructor: Chethan Kamath)

Based on lecture materials dated 19–22 Aug 2026

Abstract

These notes merge and expand on *two* source documents for Lecture 07 of CS409m. The first is a two-page handwritten proof of **Theorem 1** (stated back in Lecture 06): if G is a pseudorandom generator (PRG), then the PRG-based encryption scheme — also called the **computational one-time pad (COTP)** — is **computationally secret against eavesdroppers**. This theorem is a cornerstone result: it is exactly the “Theorem 7.1” referenced later, in Lecture 11, as the template for the one-time-pad-style distinguishing strategy used inside the IND-CPA reduction proof. The second document is the Lecture 07 *slide deck* itself, whose actual content is the main question this lecture answers: *how do we actually build a PRG?* It covers two very different answers — **practical** constructions from stream ciphers (LFSRs, Trivium, the broken CSS cipher, Salsa/ChaCha) and **theoretical** constructions from conjectured hard problems (subset-sum, integer factoring) — and the recurring lesson that *theory and practice diverge* in how PRGs are actually built and analyzed. Every diagram from both PDFs is redrawn in $\text{\TeX}$ here, and additional explanatory material is clearly marked with a ★ box.

Contents

1	Recap: What We Need Before the Theorem	3
1.1	Pseudorandom Generators (PRGs)	3
1.2	The One-Time Pad (OTP) and Perfect Secrecy	3
2	The Security Game: Computational Secrecy Against Eavesdroppers	5
3	Construction 2: PRG-Based Encryption (Stream Cipher)	6
4	Theorem 1 and Its Proof	7
4.1	Proof strategy	7
4.2	Analysis	8
5	Today’s Real Question: How Do We Actually Build a PRG?	10
5.1	PRG Is a Fundamental Primitive	10
5.2	Stream Ciphers, Formally	12
5.2.1	Recap: SKE from an LFSR	12
5.2.2	Formal syntax of a stream cipher	12

5.2.3	Stream ciphers must be unpredictable	13
5.2.4	LFSR is predictable!	14
5.3	PRG #1: Trivium = LFSRs + Non-Linearity	14
5.4	Non-Linearity Does Not Always Work: the CSS Cipher	15
5.5	Other Practical Examples: Salsa and ChaCha	16
5.6	The Theoretical Route: PRGs from Hard Problems	17
5.6.1	What is a “hard” computational problem?	17
5.6.2	PRG #2: Subset-Sum PRG	17
5.6.3	PRG #3: Factoring-Based PRG	18
6	Recap and Looking Ahead	20
7	Summary and Connections	21

1 Recap: What We Need Before the Theorem

The handwritten notes jump straight into the proof, assuming you already know two things from earlier in the course: what a **pseudorandom generator (PRG)** is, and why the **one-time pad (OTP)** is perfectly secret. We recap both here since the whole proof is a reduction that pits one against the other.

1.1 Pseudorandom Generators (PRGs)

Definition 1.1 (Pseudorandom Generator). A deterministic, polynomial-time computable function

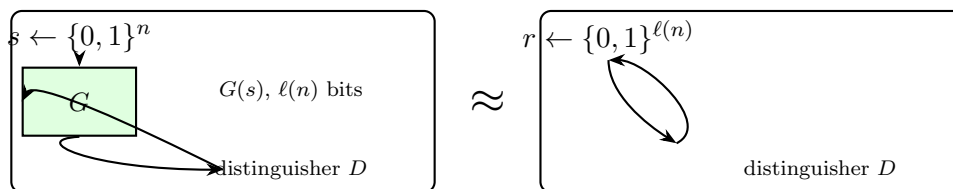
$$G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}, \quad \ell(n) > n,$$

is a **PRG** if for every PPT distinguisher D ,

$$\delta(n) := \left| \Pr_{s \leftarrow \{0, 1\}^n} [D(G(s)) = 1] - \Pr_{r \leftarrow \{0, 1\}^{\ell(n)}} [D(r) = 1] \right|$$

is negligible.

In words: G stretches a short truly-random **seed** s (length n) into a longer string $G(s)$ (length $\ell(n) > n$) that *looks* truly random to any efficient observer, even though it is completely determined by the much shorter seed s .



★ Added by Claude — Why PRGs let you “cheat” on the key-length limit

This connects directly to why we need computational (rather than perfect) security in the first place. Shannon’s theorem tells us that *perfect secrecy* (the OTP’s guarantee) is only possible if the key is at least as long as the message — there is no way around this if you demand security against an all-powerful adversary. A PRG lets us relax the requirement: instead of a truly random key as long as the message, we use a *short* truly random seed s (as long as we like, e.g. 128 bits) and *stretch* it into a long pseudorandom string $G(s)$ using the PRG. Because $G(s)$ is computationally (not perfectly) indistinguishable from true randomness, this buys us security against any *efficient* (PPT) adversary — which is the standard, realistic threat model in modern cryptography — while using a far shorter key than the message itself. This is precisely the content of the theorem proved in this lecture.

1.2 The One-Time Pad (OTP) and Perfect Secrecy

Recall from earlier in the course: for a message $m_b \in \{0, 1\}^{\ell(n)}$ (one of two messages m_0, m_1 chosen by an adversary) and a **truly random** pad $r \leftarrow \{0, 1\}^{\ell(n)}$, the ciphertext $c := r \oplus m_b$ reveals *no* information about b whatsoever — this is *perfect secrecy*. Formally, for any (even unbounded) adversary Eve and any $n \in \mathbb{N}$:

$$\Pr [\text{Eve}(r \oplus m_b) = b] = \frac{1}{2}, \quad m_0, m_1 \leftarrow \text{Eve}(1^n), \quad r \leftarrow \{0, 1\}^{\ell(n)}, \quad b \leftarrow \{0, 1\}.$$

This holds because, for a uniformly random r , the value $r \oplus m_b$ is itself uniformly random and *independent* of b — Eve's view is identically distributed whether $b = 0$ or $b = 1$.

2 The Security Game: Computational Secrecy Against Eavesdroppers

The theorem we are about to prove concerns a weaker (but still very useful) threat model than IND-CPA from Lecture 11: here the adversary (called **Eve**) only gets to *eavesdrop* on a single ciphertext, she does *not* get a chosen-plaintext encryption oracle. This is sometimes written IND*-Eav or “single-message indistinguishability in the presence of an eavesdropper”.

★ Added by Claude — The game, spelled out

1. Eve is run on input 1^n and outputs two *equal-length* messages $m_0, m_1 \in \{0, 1\}^{\ell(n)}$ of her choosing (**no** prior oracle access — this is what makes the model weaker than IND-CPA).
2. The challenger samples a seed $s \leftarrow \{0, 1\}^n$ and a bit $b \leftarrow \{0, 1\}$, computes the ciphertext $c := G(s) \oplus m_b$, and hands c to Eve.
3. Eve outputs a guess b' , and **wins** if $b' = b$.

Construction 2 is *computationally secret against eavesdroppers* if every PPT Eve wins with probability at most $\frac{1}{2} + \text{negl}(n)$.

The slides frame this same threat model as a checklist worth memorizing, splitting it into an **attack model** (what Eve is allowed to do) and a **security goal** (what we demand of the scheme):

Attack Model: Eavesdropping	Security Goal: Secrecy, w.h.p.
1. Eve is <i>efficient</i>, i.e. PPT. 2. Knows the description of the SKE (Kerckhoffs’s principle). 3. The shared key is hidden from Eve. 4. Can eavesdrop and learn the ciphertext.	• Eve learns <i>no information</i>, except with negligible probability. • Formalized as adversarial indistinguishability — exactly the game above.

★ Added by Claude — Kerckhoffs’s principle

Point 2 in the attack model — “Eve knows the description of the SKE” — is a foundational assumption in cryptography called **Kerckhoffs’s principle** (1883): *a cryptosystem should be secure even if everything about it, except the key, is public knowledge*. This is the opposite of “security through obscurity”. It’s why we always analyze schemes assuming the adversary knows Gen, Enc, Dec exactly (even the PRG G itself!) — the *only* secret is the key/seed s . Real systems that violate this (hiding a proprietary, unpublished algorithm instead of relying on a secret key) have a long history of turning out to be broken once reverse-engineered — exactly what happens to the CSS stream cipher later in these notes.

3 Construction 2: PRG-Based Encryption (Stream Cipher)

Construction 3.1 (Construction 2). Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ be a PRG. Define $\Sigma = (\text{Gen}, \text{Enc}, \text{Dec})$ for message space $\mathcal{M} = \{0, 1\}^{\ell(n)}$ as:

- $\text{Gen}(1^n)$: sample seed $s \leftarrow \{0, 1\}^n$ (this is the secret key).
- $\text{Enc}_s(m)$: output $c := G(s) \oplus m$.
- $\text{Dec}_s(c)$: output $m := G(s) \oplus c$.

★ Added by Claude — This is literally how stream ciphers work

Construction 2 is the textbook definition of a (synchronous) stream cipher: expand a short key into a long pseudorandom keystream using G , then XOR the keystream with the message. This is precisely the same idea behind the OFB and CTR modes of Lecture 11 — there, a PRF F is used as (part of) a PRG to generate the keystream block-by-block, instead of a single monolithic G . So Construction 2 here is really the “single-shot”, simplest-possible ancestor of the multi-block, IV-based modes you saw later in the course. Real-world stream ciphers like RC4, Salsa20/ChaCha20 all follow exactly this template, differing only in how G (the keystream generator) is built.

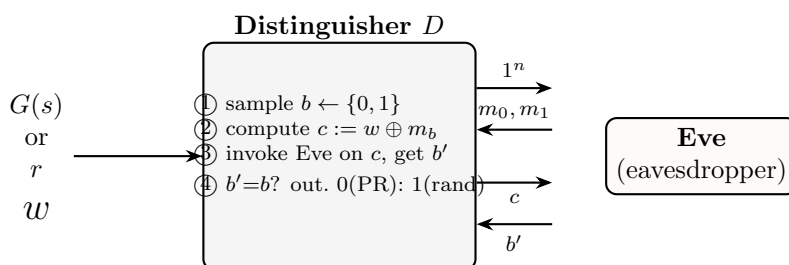
4 Theorem 1 and Its Proof

Theorem 4.1 (Theorem 1, stated in Lecture 06 and proved here). *Assuming G is a PRG, Construction 2 (equivalently, the computational one-time pad, COTP) is computationally secret against eavesdroppers.*

4.1 Proof strategy

Assume for contradiction that there exists an eavesdropper **Eve** that breaks the secrecy of Construction 2 (with non-negligible probability). Recall the general reduction strategy from the lecture: given a subroutine Eve that breaks the scheme, we construct a **distinguisher** D for the PRG G . D is the so-called *reduction algorithm*: it uses Eve as a black-box subroutine to distinguish $G(s)$ (for random seed s) from a truly random string r .

Schematic diagram of the reduction



Pseudocode for $D(w, 1^n)$

- 1) Obtain (m_0, m_1) by invoking subroutine $\text{Eve}(1^n)$.
- 2) Sample $b \leftarrow \{0, 1\}$ and compute $c := w \oplus m_b$.
- 3) Invoke Eve on c and obtain guess b' .
- 4) Output 0 (“pseudorandom”) if $b = b'$; otherwise output 1 (“random”).

★ Added by Claude — Reading the pseudocode

D receives an $\ell(n)$ -bit string w that is *either* $G(s)$ for a random seed s , *or* a truly random string r — and D ’s whole job is to guess which. D ’s trick is to treat w as if it were the PRG output inside a simulated run of the Construction-2 secrecy game, using Eve as the adversary in that simulated game. If w really is $G(s)$, this simulation is *exactly* the real security game against Construction 2 (so Eve should do noticeably better than random guessing, by assumption). If w is truly random r , then $c = r \oplus m_b$ is an actual one-time pad, so by perfect secrecy Eve can do no better than random guessing, no matter how powerful she is. D exploits this gap by outputting 0 exactly when Eve’s guess b' was correct.

4.2 Analysis

Since Eve is assumed to break the secrecy of Construction 2 with **non-negligible probability**, there exists a polynomial p such that, for *infinitely many* n 's,

$$\Pr [\text{Eve}(G(s) \oplus m_b) = b] - \frac{1}{2} \geq \frac{1}{p(n)}, \quad (1)$$

where the probability is over $m_0, m_1 \leftarrow \text{Eve}(1^n)$, $s \leftarrow \{0, 1\}^n$, $b \leftarrow \{0, 1\}$.

★ Added by Claude — Why “infinitely many n ” rather than “all n ”?

This is a subtlety worth pausing on. The *definition* of a negligible function $\text{negl}(n)$ is one that is eventually smaller than $1/p(n)$ for *every* polynomial p and *all sufficiently large* n . Logically negating “Eve’s advantage is negligible” therefore does *not* give “Eve’s advantage is non-negligible for all n ” — it only gives “there exists *some* polynomial p such that Eve’s advantage exceeds $1/p(n)$ for *infinitely many* n ”. This is a strictly weaker (and correct!) statement, and it’s exactly what’s needed: the proof only has to derive a contradiction using those infinitely-many n ’s where Eve’s advantage is large, which is exactly what the last line of the proof does (“for the infinitely-many n ’s guaranteed above”). This is a common and easy-to-miss subtlety in cryptographic proofs by contradiction — get the direction of the negation of “negligible” right!

On the other hand, by the (perfect) security of the OTP, for *all* $n \in \mathbb{N}$,

$$\Pr [\text{Eve}(r \oplus m_b) = b] = \frac{1}{2}, \quad (2)$$

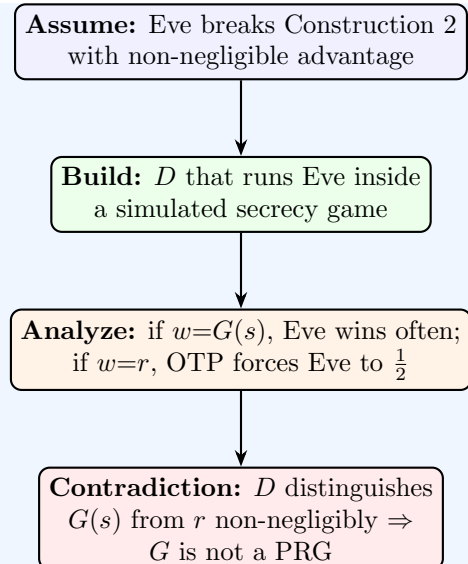
where the probability is over $m_0, m_1 \leftarrow \text{Eve}(1^n)$, $r \leftarrow \{0, 1\}^{\ell(n)}$, $b \leftarrow \{0, 1\}$.

Now note that our distinguisher D outputs 0 **if and only if** Eve correctly guesses b (i.e. $b' = b$) in the reduction. Therefore, for the infinitely-many n ’s guaranteed above, we get:

$$\begin{aligned} \left| \Pr_{s \leftarrow \{0,1\}^n} [D(G(s)) = 0] - \Pr_{r \leftarrow \{0,1\}^{\ell(n)}} [D(r) = 0] \right| &= \left| \Pr[\text{Eve}(G(s) \oplus m_b) = b] - \Pr[\text{Eve}(r \oplus m_b) = b] \right| \\ &\geq \left| \left(\frac{1}{2} + \frac{1}{p(n)} \right) - \frac{1}{2} \right| \quad [\text{by (1) and (2)}] \\ &= \frac{1}{p(n)}. \end{aligned}$$

This **contradicts the pseudorandomness of G** (which requires D ’s distinguishing advantage to be negligible for *all* PPT D , so it cannot be $\geq 1/p(n)$ infinitely often). Thus, such an Eve cannot exist. ■

★ Added by Claude — The proof in one picture



This “assume the scheme breaks $\rightarrow$ build a reduction $\rightarrow$ derive a contradiction with the underlying primitive’s security” skeleton is the *exact same* template used for the (more complex) IND-CPA proof in Lecture 11 — in fact the Lecture 11 notes explicitly say the PRF-distinguisher’s challenge-query strategy is copied from “the proof of Theorem 7.1 (LOTP)”, i.e. precisely this theorem. The only real difference in Lecture 11 is that (a) the primitive is a PRF instead of a PRG, accessed via an oracle rather than given as a single string, and (b) the adversary gets many encryption queries (IND-CPA) rather than none (eavesdropper-only), which is why Lecture 11 needed the extra “Claim 2” case analysis on repeated random indices — a complication that simply does not arise here, since Eve only ever sees a *single* ciphertext.

5 Today’s Real Question: How Do We Actually Build a PRG?

Everything so far (Theorem 1 and its proof) *assumed* we already had a PRG G in hand. The rest of Lecture 07 — the slide deck — addresses the natural next question: **where does G actually come from?** The slides pursue two parallel, very different answers:

Practical: Stream Ciphers	Theoretical: From Hard Problems
Fast, deployed constructions (LFSRs, Trivium, Salsa/ChaCha) with <i>no proof</i> of security — just an absence of known attacks.	Constructions with a <i>security proof</i> , but which reduce security to unproven (conjectured) computational hardness assumptions (e.g. factoring, subset-sum).

★ Added by Claude — The big takeaway, previewed

The slides’ own tagline for this lecture is: “*things work differently in theory and practice.*” Keep this in mind as you read both halves below: the **practical** constructions are blazingly fast (essential for real-time encrypted traffic) but their security rests entirely on cryptanalysts having *failed* to break them so far — there is no proof, only survived attacks. The **theoretical** constructions come with an actual security *reduction* (in the same “assume broken $\Rightarrow$ contradiction” style as Theorem 1 above), but are typically far too slow for practical deployment, since they perform expensive modular arithmetic per output bit rather than cheap bit operations. Modern practice therefore builds *fast, standardized, heavily-scrutinized* primitives (like AES, ChaCha20) and treats them as PRGs/PRFs *by assumption* (a “heuristic” security stance), while the theoretical constructions mostly serve to demonstrate that PRGs *can* be built rigorously from cleaner, more general assumptions (and are indispensable in more advanced constructions later in the course, and in cryptography research generally).

5.1 PRG Is a Fundamental Primitive

The slides open this half of the lecture with a cautionary real-world story: since randomness for encryption is so often generated using PRGs, a **backdoored PRG breaks everything built on top of it**. The canonical example is **Dual_EC_DRBG**, a PRG standardized by NIST that was later shown (and, per leaked NSA documents revealed by Edward Snowden as part of the **Bullrun** program) to likely contain a secret backdoor allowing the NSA to predict its output given only a small amount of observed keystream. NIST formally withdrew its recommendation of Dual_EC_DRBG in 2014.

★ Added by Claude — Why a backdoored PRG is catastrophic

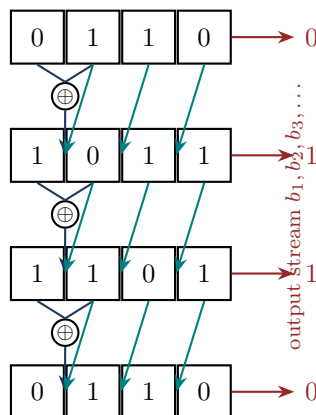
If a PRG used to generate an encryption key (or an IV, or a seed for any of the modes of operation you saw in Lecture 11!) is secretly predictable to whoever planted the backdoor, then *every single message encrypted using that randomness* becomes readable to the backdoor’s holder — no matter how strong the encryption algorithm itself is. This is exactly why the **provenance and public scrutiny of randomness sources** matters so much in practice, and why standards bodies (and this course!) place such emphasis on PRGs having a clear,

checkable security argument (or, failing that, extensive public cryptanalysis) rather than being adopted on trust.

5.2 Stream Ciphers, Formally

5.2.1 Recap: SKE from an LFSR

Before formalizing, recall the running example: a **Linear Feedback Shift Register (LFSR)**. Below is a 4-bit LFSR with *tap* at position 2 (i.e. the feedback bit is the XOR of two specific cells of the register).

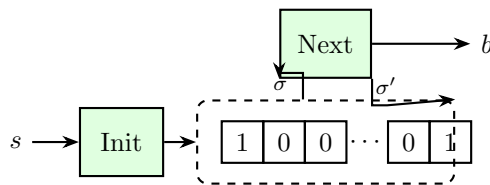


The construction recalled: state = (0, 1, 1, 0); at each step, output the leftmost bit, shift everything left, and feed the XOR of two tapped cells (here, the two leftmost) into the vacated rightmost cell. The secrecy game (recall from earlier lectures) is: SKE simply **XORs the LFSR's output stream with the message bits**, exactly like Construction 2's PRG-based encryption, but now with the LFSR playing the role of a (candidate) PRG.

5.2.2 Formal syntax of a stream cipher

Definition 5.1 (Stream Cipher — Syntax). A stream cipher Σ is a pair of *deterministic* algorithms (Init, Next):

- Init takes as input a seed $s = s_1 \| \dots \| s_n$ and outputs an initial state σ_0 .
- Next, on input the current state σ , outputs the next state σ' and an output bit b .



Exercise 5.2 (Exercise 1, slides). Formally describe the n -bit LFSR with taps $t_1, \dots, t_\ell$.

★ Added by Claude — Solution

State $\sigma = (s_1, \dots, s_n) \in \{0, 1\}^n$. $\text{Init}(s) := s$ (the seed *is* the initial state). Given taps $t_1, \dots, t_\ell \in [1, n]$:

$$\text{Next}(s_1, \dots, s_n) := \left((s_2, \dots, s_n, f), s_1 \right), \quad \text{where } f := s_{t_1} \oplus s_{t_2} \oplus \dots \oplus s_{t_\ell}.$$

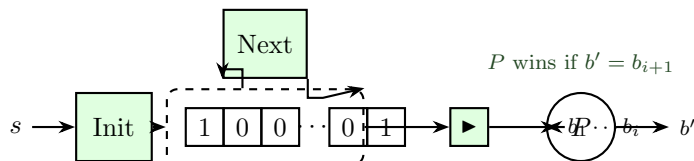
i.e. output the leftmost bit s_1 , shift all bits one position left, and feed the XOR of the tapped

bits into the newly-vacated rightmost cell.

5.2.3 Stream ciphers must be unpredictable

Informally: seeing output bits $b_1, \dots, b_i$ should not help in predicting b_{i+1} .

Definition 5.3 ((Next-Bit) Unpredictability). A stream cipher $\Sigma = (\text{Init}, \text{Next})$ is **unpredictable** if for every PPT predictor P , the probability of P winning the following game, away from half, is negligible: run $\text{Init}(s)$ for random seed s , iterate Next to produce $b_1, \dots, b_i$ for a P -chosen i , give P these bits, and P wins if its guess b' equals the *next* bit b_{i+1} .



Exercise 5.4 (Exercise 2, slides). Extend LFSR-based SKE to *any* stream cipher, and prove its security.

★ Added by Claude — Solution sketch

Exactly as for the LFSR: define $G_\Sigma(s) := b_1 \| b_2 \| \dots \| b_{\ell(n)}$, where b_i is the i -th output bit produced by repeatedly running Next starting from $\text{Init}(s)$. Then set $\text{Enc}_s(m) := G_\Sigma(s) \oplus m$, exactly as in Construction 2. The security proof is a *hybrid argument* over the $\ell(n)$ output bits (structurally identical to the “Exercise 1” hybrid argument you already solved for the Lecture 11 baseline mode!): if some adversary could distinguish $G_\Sigma(s)$ from a truly random string, then by a standard hybrid/averaging argument there must be some index i at which the adversary effectively distinguishes “the real bit b_i following true-random $b_1, \dots, b_{i-1}$ ” from “a fresh random bit following the same prefix” — which is exactly a next-bit predictor for Σ . This is precisely the content of **Fact 2** below.

Claim 5.5 (Fact 2, slides). *Unpredictability is equivalent to pseudorandomness.*

★ Added by Claude — Why this equivalence is so useful

This is a famous and extremely useful theorem (due to Yao). It means that to check a candidate PRG’s security, you only ever need to rule out *next-bit prediction* — a much more concrete, checkable property — rather than reasoning about *all* possible statistical tests a distinguisher could run. In practice, cryptanalysts attacking stream ciphers are, in effect, always searching for a next-bit predictor (or a full state-recovery attack, which trivially implies one) — exactly what happens to the plain LFSR next.

5.2.4 LFSR is predictable!

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ & & & \vdots & & & & \end{bmatrix} \times \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_7 \\ s_8 \end{bmatrix} = \begin{bmatrix} b_9 \\ b_{10} \\ b_{11} \\ \vdots \\ b_{15} \\ b_{16} \end{bmatrix}$$

Recall that for a (standard) n -bit LFSR, $b_1 \parallel \dots \parallel b_n = s_1 \parallel \dots \parallel s_n$, and b_{n+1} can be generated from $b_n, \dots, b_1$ (it's just the next application of Next). **What if we “fix” this by starting to stream only from b_{n+1} onward**, hiding the raw seed bits? **Still predictable!** Given $b_{n+1}, \dots, b_{2n}$, it is possible to generate b_{2n+1} . **Observation:** since the LFSR is *linear*, we end up with a system of n linear equations in n unknowns $s_1, \dots, s_n$ (as sketched above), which can be solved by **Gaussian elimination** in $\approx n^5$ time.

Attack 1 (Seed-recovery for a general LFSR).

Input: bit sequence $\bar{b} := (b_{n+1}, \dots, b_{2n})$.

Output: seed s .

Algorithm: (1) construct the coefficient matrix $\bar{C}$ for the LFSR; (2) solve $\bar{C} \cdot s = \bar{b}$ via Gaussian elimination.

Underlying issue: **linearity**, which usually leads to attacks. **Solution:** introduce **non-linear** feedback!

★ Added by Claude — A concrete tiny worked example

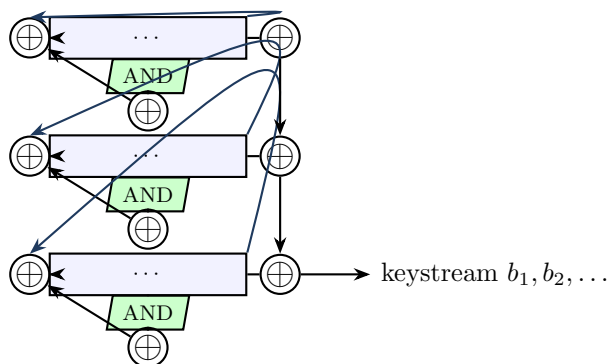
Take the 4-bit LFSR from the recap diagram above, taps at positions 1 and 2, seed $s = (0, 1, 1, 0)$. Running it gives the output stream $b_1 b_2 b_3 b_4 \dots = 0, 1, 1, 0, \dots$ (matching the recap figure). Suppose an attacker sees only b_5, b_6, b_7, b_8 (*after* the raw seed has scrolled out). Since $b_{i+4} = b_i \oplus b_{i+1}$ (the tap-1,2 recurrence), knowing 4 consecutive output bits anywhere in the stream lets you write down 4 linear equations relating them back to $s_1, \dots, s_4$ (by running the recurrence backwards), and solve for s by Gaussian elimination — exactly Attack 1. This is why real stream ciphers *never* use a bare LFSR as the direct output: they use it only as an *internal, hidden* building block, combined non-linearly, as in Trivium next.

Faster than Gaussian elimination: in fact, for LFSRs specifically, there's an even faster classical algorithm — the **Berlekamp–Massey algorithm** — which recovers the shortest LFSR (and hence its seed/taps) generating a given bit sequence in just $O(n^2)$ time, using only $2n$ consecutive output bits, rather than the $O(n^5)$ generic Gaussian-elimination bound quoted in the slides. This makes plain LFSRs even more thoroughly broken as stream ciphers than the slides' bound alone suggests.

5.3 PRG #1: Trivium = LFSRs + Non-Linearity

Construction 5.6 (Construction 1, slides — Trivium, schematic). *Three coupled shift registers, each of which (a) feeds a **non-linear** (AND-gate) combination of two of its own internal bits back*

into its own feedback, and (b) contributes its output, XORed together with the other two registers' outputs, to the final keystream bit.



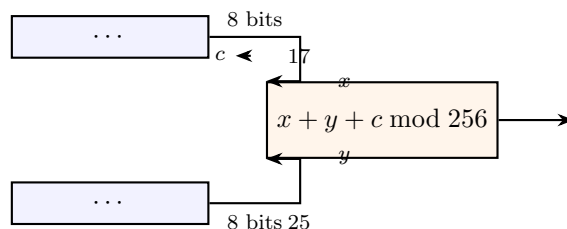
Security: the best known attack is brute-force search over the internal state, which takes **exponential** time. **Caution** (emphasized in the slides): *absence of known attacks is not a proof of security* — this is the “practical” branch’s fundamental limitation, in contrast to the reduction-based proofs of the “theoretical” branch.

★ Added by Claude — Trivium in the real world

Trivium is a real, standardized stream cipher, one of the winners of the **eSTREAM** project (a EU-funded competition, 2004–2008, to identify stream ciphers suitable for widespread adoption, run in the aftermath of several broken designs). It is deliberately simple (just three interlinked shift registers and a handful of AND/XOR gates) and extremely fast/compact in hardware, making it popular for constrained devices (RFID tags, low-power sensors). Two decades of public cryptanalysis have found no attack faster than brute force on full Trivium, which is about as close to “confidence” as the practical, no-proof branch of PRG design ever gets.

5.4 Non-Linearity Does Not Always Work: the CSS Cipher

Construction 5.7 (Construction 2, slides — CSS schematic). *Two LFSRs of lengths 17 and 25 bits, each outputting 8 bits per step, combined via integer addition modulo 256 with a carry bit c fed back into the next step.*



Real-world example: the **Content-Scrambling System (CSS)**, used to prevent piracy of DVDs. It relied on a 40-bit stream cipher (two LFSRs plus this modulo-256 addition). Despite the non-linear addition/carry step, CSS could be broken in $\approx 2^{16}$ time given only 100 bits of output — non-linearity alone is **not** sufficient for security if the non-linear operation is too weak or too small in scale relative to the state.

★ Added by Claude — Why did non-linearity fail here?

CSS’s fatal flaw is that its *overall* key space is only 40 bits (deliberately weakened at design time due to 1990s US export restrictions on cryptography!), and worse, the non-linear addition-with-carry is actually *much weaker* than it looks: because addition mod 256 with carry propagation can be locally approximated by simpler linear/algebraic relations, and because one of the two LFSRs is short (17 bits) and can be brute-forced directly, an attacker can guess the short LFSR’s state outright (2^{17} guesses), then use the algebraic structure of the carry-adder to recover the longer LFSR’s state from a modest amount of output — landing at the quoted $\approx 2^{16}$ total. CSS was practically broken by 1999 (the DeCSS program), triggering a wave of DVD-industry lawsuits and becoming one of the most famous cautionary tales in applied cryptography: *export-restricted, short keys, and under-analyzed non-linear combiners are a recipe for disaster, even when the design intuitively “looks” non-linear enough.*

5.5 Other Practical Examples: Salsa and ChaCha

The slides briefly showcase two more modern, widely-deployed stream ciphers — **Salsa20** and its variant **ChaCha20** (both designed by Daniel J. Bernstein) — each built from repeated rounds of modular addition, XOR, and bit rotation (an “ARX” design: Add–Rotate–XOR) applied to a 4×4 grid of 32-bit words, run for typically 20 rounds. The slides’ one-line takeaway: these run *faster in software* than Trivium (which was optimized more for compact hardware implementations).

★ Added by Claude — Where you’ve actually used ChaCha20

ChaCha20 (often paired with the Poly1305 MAC, as “ChaCha20-Poly1305”) is one of the two cipher suites (alongside AES-GCM) permitted in modern TLS 1.3, and is the default cipher in the WireGuard VPN protocol. It was specifically favored on mobile devices lacking hardware AES acceleration, since its pure ARX design runs efficiently in software without needing dedicated silicon — unlike AES, whose fast software implementations rely on hardware AES-NI instructions.

5.6 The Theoretical Route: PRGs from Hard Problems

5.6.1 What is a “hard” computational problem?

The slides contrast **easy** problems — ones with known efficient (polynomial-time) algorithms, such as matrix multiplication ($\approx n^3$ steps) or solving a system of linear equations ($\approx n^5$ time, i.e. exactly the Gaussian elimination used to break LFSRs above!) — with problems believed to have *no* efficient algorithm, such as Sudoku or Chess. The natural idea, *build a PRG whose security reduces to a hard problem like chess*, runs into a real obstacle: **problems like chess and Sudoku are “too hard”** to build efficient PRG constructions from (there’s no known way to embed a short seed into a chess position such that solving the position reveals structured information about the seed, in a way useful for building a PRG). The fix: work instead with **moderately-hard** problems, typically sourced from number theory, geometry, or (more recently) machine learning, including:

1. Integer factorization,
2. The subset-sum problem,
3. Learning with errors (LWE).

Why do we believe these problems are hard? The slides’ honest answer: *for a long time, smart people have tried to find efficient algorithms for them, and failed.* This is exactly the same “absence of known attacks” epistemic stance as the practical/stream-cipher branch — the difference is that here, the *scheme built on top* of the hard problem comes with a rigorous *reduction*, so all the residual uncertainty is concentrated in one, cleanly-stated, widely-studied assumption, rather than spread across an ad-hoc cipher design.

★ Added by Claude — Why LWE is worth remembering

Learning With Errors (LWE), listed as the third example, deserves a special mention: unlike factoring and subset-sum (both of which are broken by Shor’s algorithm on a large-enough quantum computer!), LWE is currently believed to be hard even for quantum computers. This is exactly why LWE-based constructions underlie essentially all of the **post-quantum cryptography** standards recently finalized by NIST (e.g. ML-KEM/Kyber, ML-DSA/Dilithium) — the entire push to migrate the internet’s cryptography to “quantum-safe” algorithms is, in large part, a push toward LWE-based (and closely related lattice-based) primitives, in place of factoring/discrete-log-based ones like RSA and elliptic-curve cryptography.

5.6.2 PRG #2: Subset-Sum PRG

Problem 3 (Subset-Sum).

Input: a prime modulus q and integers $a_1, \dots, a_n < q$.

Solution: a set $\mathcal{I} \subseteq [1, n]$ such that $\sum_{i \in \mathcal{I}} a_i \equiv 0 \pmod{q}$.

Believed to be hard for (large enough) $n \approx \log(q)$. The fastest known algorithms require (exponential) time $\approx q$.

Claim 5.8 (Assumption 1, slides — Subset-sum assumption, informal). *When $q, a_1, \dots, a_n$ are sampled at random, the subset-sum problem is hard to solve for PPT algorithms.*

Construction 5.9 (Construction 3, slides — Subset-sum PRG). *For $q, a_1, \dots, a_n$ as in Assumption*

1, with $q > 2^n$:

$$G_{q,a_1,\dots,a_n}(s_1 \| \dots \| s_n) := \sum_{i=1}^n s_i a_i \pmod{q}.$$

★ Added by Claude — Why this is a PRG, informally, and a cautionary history

The seed is n bits ($s_1, \dots, s_n \in \{0, 1\}$), and the output is a single element of $\mathbb{Z}_q$, needing $\log_2 q > n$ bits to write down — so G is indeed length-*expanding*, as a PRG must be. Intuitively, $\sum_i s_i a_i \pmod{q}$ looks like a “random-enough” linear combination of the (public, random) a_i ’s, and recovering s from the output is *exactly* an instance of the subset-sum problem in disguise (a linear *knapsack*-style problem) — specifically the related *subset-sum decision/search* problem, closely connected to what’s historically called the **knapsack problem**. This connection is a double-edged sword: knapsack-based cryptosystems (most famously the **Merkle–Hellman cryptosystem**, 1978) were, for a time, seen as an attractive alternative to RSA precisely because subset-sum *felt* hard — until Adi Shamir showed in 1982 that the specific “trapdoored”, low-density knapsacks used in Merkle–Hellman could be broken efficiently using lattice-basis-reduction techniques. This is exactly why the slides are careful to restrict to *randomly sampled*, suitably high-density a_i ’s (as in Assumption 1) — the security of subset-sum-based constructions is extremely sensitive to exactly how the instance is generated, a recurring theme in hardness-assumption-based cryptography.

5.6.3 PRG #3: Factoring-Based PRG

Problem 4 (Integer Factorization).

Input: an integer $N := q_1 q_2$, where q_1, q_2 are primes.

Solution: q_1 (equivalently, q_2).

Believed to be hard when q_1, q_2 are random primes of roughly the same bit-length. The fastest known algorithm (the general number field sieve) requires **sub-exponential** time $\approx N^{1/3}$ (up to polylog factors) — notably *faster* than the exponential-time bound for subset-sum above, which is why factoring-based schemes in practice need substantially larger parameters (N typically ≥ 2048 bits) than, say, symmetric ciphers of comparable security.

Claim 5.10 (Assumption 2, slides — Factoring assumption, informal). *Factoring N sampled as above is hard for PPT algorithms.*

Construction 5.11 (Construction 4, slides — Factoring-based stream cipher). *Stream cipher parameter: N as in Assumption 2.*

- Init: *initial state* $x_0 \leftarrow [0, N - 1]$.
- Eval (*i.e.* Next): on input x_i , compute $x_{i+1} := x_i^2 \pmod{N}$, and output the least significant bit (LSB) of x_i .

★ Added by Claude — This is the Blum Blum Shub (BBS) generator

Construction 4 is a direct presentation of the famous **Blum Blum Shub (BBS) pseudo-random generator** (1986). Its security reduces (via a proof in the same style as Theorem 1, though more involved) to the hardness of factoring N — more precisely to the closely related

quadratic residuosity problem modulo N . It is a beautiful illustration of the theoretical branch's core trade-off from the box at the start of this section: BBS comes with an actual security *proof* tied to a well-studied hardness assumption, but it is glacially slow compared to Trivium or ChaCha20 — repeated squaring modulo a large N (a few hundred to a few thousand bits) is vastly more expensive per output *bit* than a handful of XOR/shift/AND operations, and BBS as shown here outputs only *one* bit per squaring! (Variants that output $O(\log \log N)$ bits per iteration, proven secure under the same assumption, exist and help somewhat, but the fundamental gap to symmetric-primitive speeds remains.) This is precisely why, in practice, no widely deployed system uses BBS directly for bulk encryption — but constructions like it remain foundational for building other primitives (e.g. public-key encryption) where you need the security *proof* far more than you need raw speed.

6 Recap and Looking Ahead

To recap: we **constructed PRGs** (and hence, via Theorem 1, computational one-time pads/COTP) two ways:

- **Theoretical** constructions from hardness assumptions (subset-sum PRG, factoring-based/BBS PRG) — each with an actual security *proof*, but comparatively slow.
- **Practical** constructions from stream ciphers (LFSRs $\rightarrow$ broken by linearity; Trivium, Salsa/ChaCha $\rightarrow$ fast, unbroken in practice, but with *no* security proof, only surviving cryptanalysis).

Take-away: theory and practice genuinely diverge in how PRGs get built and trusted.

Next lecture tackles a natural follow-up question: our PRGs above stretch a seed to *some fixed* longer length $\ell(n)$ — but what if we need to encrypt *multiple* messages, or messages of varying/larger length? This needs:

- **Length extension of a PRG** — stretching a PRG’s output even further, from $\ell(n)$ bits to an arbitrary polynomial length, while preserving pseudorandomness.
- A powerful new proof technique: the **hybrid argument** — which you in fact already previewed and used yourself, in the Lecture 11 notes, to prove that independently encrypting each block of a long message preserves IND-CPA security (Exercise 1 there)! That hybrid argument is a direct descendant of the length-extension hybrid argument introduced next lecture.

Sources (as cited in the slides)

1. §3.2 and §3.3 in Katz & Lindell, *Introduction to Modern Cryptography* (3rd ed.), 2014, for more on computational secrecy and the computational OTP.
2. §7.1.1 in the same reference, for more on LFSRs.
3. §3.8 of Dan Boneh and Victor Shoup, *A Graduate Course in Applied Cryptography*, Version 0.6, 2023, for more on the cryptanalysis of CSS.
4. The Wikipedia entry on the **Bullrun** program, for more on the Dual_EC_DRBG backdoor story.

★ Added by Claude — A fun aside from the slides

The slides end with the classic *xkcd* #1210 comic (“Random Number”) as a lighthearted reminder of what *not* to do for randomness generation — shouting “monkey tacos, I’m so random!” is not a valid PRG, no matter how much you might will it to be pseudorandom. On a more practically useful note, the slides also point to how **Minecraft** procedurally generates entire game worlds from a single 48- or 64-bit seed using simple generators like a Linear Congruential Generator (LCG, related to the taps-and-shifts ideas here) or xorshift — explaining why sharing just a short numeric “seed” with a friend reproduces an entire, deterministic, enormous game world. (There are even community-curated databases of seeds that generate particularly interesting or resource-rich Minecraft worlds!)

7 Summary and Connections

- We defined **computational secrecy against eavesdroppers**: a single-challenge, no-oracle relaxation of the IND-CPA game from later lectures.
- We defined **Construction 2**, the PRG-based stream cipher: $\text{Enc}_s(m) = G(s) \oplus m$.
- We proved **Theorem 1**: if G is a PRG, Construction 2 achieves this notion of secrecy, via a **reduction** that turns any eavesdropper Eve who breaks the scheme into a distinguisher D that breaks the PRG G — contradiction.
- The proof technique — *assume the scheme is broken, build a reduction using the attacker as a subroutine, derive a contradiction with the underlying primitive's security* — is the master template reused (in more elaborate form) throughout the rest of the course, most visibly in the Lecture 11 IND-CPA proof for the PRF-based scheme, which explicitly borrows this theorem's OTP-based distinguishing strategy for its own challenge-query step.

★ Added by Claude — Quick self-check questions

1. Why does the reduction D need to invoke Eve *twice* conceptually — once to get (m_0, m_1) and once to get b' — and why is this legitimate (i.e. why can D treat Eve as a black box it controls)?
2. Where exactly in the analysis is the assumption “ G is a PRG” used, and where is the assumption “the OTP is perfectly secret” used? (Hint: look at which of equations (1) and (2) needs which assumption.)
3. Why is it crucial that Eve gets to see only *one* ciphertext in this game, unlike in the IND-CPA game of Lecture 11? What would go wrong with the proof if Eve got an encryption oracle before the challenge?
4. Convince yourself that D runs in polynomial time whenever Eve does (this is required for D to be a legitimate PPT PRG-distinguisher).
5. Why does a *non-negligible* advantage for Eve only need to be exhibited for *infinitely many* n (not all n) in order to contradict G being a PRG?